

TaskFlow — Backend Engineer Take-Home Assignment

Stack:

FastAPI or Django REST Framework (your choice) · PostgreSQL · Redis · Python 3.11+

Overview

Build a small **Task Management API with Notifications** ("TaskFlow"). Users create projects, assign tasks, and receive notifications (simulated — no real email/SMS needed) when a task's due date passes or its status changes. This assignment intentionally spans several areas of backend engineering — API design, background processing, caching, and deployment — because we want to see how you prioritize and make tradeoffs under a real time constraint, not just whether you can complete a checklist.

There is more here than most people can polish end-to-end. That's by design. We'd rather see a few areas done thoughtfully with clear reasoning about what was deprioritized, than everything done at a shallow, average level.

Functional Requirements

1. Authentication

- User signup and login (JWT or session-based — your choice, be ready to explain why).
- Passwords must be properly hashed (bcrypt/argon2), never logged or returned in API responses.

2. Projects & Tasks

- Full CRUD for projects.
- Tasks belong to a project and have: **status** (todo / in_progress / done), **assignee**, and **due_date**.
- A user should not be able to view or edit another user's project (authorization, not just authentication).

3. Notifications

- When a task's due date passes without it being marked done, or when a task is reassigned, a notification record should be created.
- Notification creation must run via a **background job** (Celery, RQ, Django-Q, or equivalent) — not synchronously inside the request/response cycle.
- Delivery can be simulated (e.g. write to a log or a notifications table) — no real email/webhook integration required.

4. Search & Filtering

- **GET /tasks** should support filtering by status, assignee, and due date range, with pagination.

- This endpoint should be cache-backed (Redis), with a sensible invalidation strategy on task updates — stale reads after a status change will be treated as a bug.

5. Operational Endpoints

- **GET /health** — health check endpoint.
- **GET /metrics** — basic request/error counters, or a full Prometheus-compatible endpoint if you want to go further.

Deliverables

- A Git repository (zip or link) with an incremental commit history — we're interested in how you work, not just the final diff.
- A working **Dockerfile** and **docker-compose.yml** that bring up the full stack (app, database, Redis, background worker) with a single command.
- A **CI pipeline config** (GitHub Actions, GitLab CI, or equivalent) running at minimum lint + tests. It doesn't need a live green run — the config itself is the deliverable.
- Either a **live deployed URL** (any free-tier host is fine — Render, Railway, Fly.io, a VM, etc.) **or**, if you'd rather not host it, a documented/scripted deployment path (a deploy script, Terraform/Ansible snippet, etc.) that shows the steps even if not executed. Please state clearly in your README which path you took.
- A **README** covering: setup instructions that work from a clean clone, a short architecture overview, and a "tradeoffs & what I'd do with more time" section. This last section is genuinely part of what we're evaluating — be candid.
- A test suite covering the core flow and, ideally, the trickier paths (authorization boundaries, cache invalidation, background job triggering).

Questions about scope or ambiguous requirements? Use your best judgment and document the assumption in your README — that's a normal and expected part of real engineering work. If something is genuinely blocking, feel free to reach out.